

clean-css logo			
1	2	3	4
8 - What's new in version 4.1 - Important: 4.0 breaking changes - Constructor options - Compatibility modes - Fetch option - Formatting options - Inlining options - Optimization levels - Level 0 optimizations - Level 1 optimizations - Level 2 optimizations - Plugins	7 - clean-css is a fast and efficient CSS optimizer for <u>Node.js</u> platform and <u>any modern browser</u>. - According to <u>tests</u> it is one of the best available. Table of Contents - Node.js version support - Install - Use - What's new in version 5.3 - What's new in version 5.0 - What's new in version 4.2	9	5
- Minify method - Promise interface - CLI utility - FAQ - How to optimize multiple files? - How to process multiple files without concatenating them into one output file? - How to process remote @imports correctly? - How to apply arbitrary transformations to CSS properties? - How to specify a custom rounding precision?	- How to keep a CSS fragment intact? - How to preserve a comment block? - How to rebase relative image URLs? - How to work with source maps? - How to apply level 1 & 2 optimizations at the same time? - What level 2 optimizations do? - What errors and warnings are? - How to use clean-css with build tools? - How to use clean-css from web browser? - Contributing - How to get started?	- Acknowledgments - License	Node.js version support clean-css requires Node.js 10.0+ (tested on Linux, OS X, and Windows)
9	10	11	12
16 What's new in version 5.0 clean-css 5.0 introduced some breaking changes: - Node.js 6.x and 8.x are officially no longer supported; - transform callback in level-1 optimizations is removed in favor of new <u>plugins</u> interface; - changes default Internet Explorer compatibility from 10+ to >11, to revert the old default use { compatibility: 'ie10' } flag;	15 What's new in version 5.3 clean-css 5.3 introduces one new feature: variables can be optimized using level 1's variableValueoptimizers option, which accepts a list of <u>value optimizers</u> or a list of their names, e.g. variableValueoptimizers: ['color', 'fraction'].	14 Use <pre>var CleanCSS = require('clean-css'); var input = '{font-weight:bold;}'; var options = { /* options */ }; var output = new CleanCSS(options).minify(input);</pre>	13 Install <pre>npm install --save-dev clean-css</pre>

- changes default rebase option from true to false so URLs are not rebased by default. Please note that if you set rebaseTo option it still counts as setting rebase: true to preserve some of the backward compatibility. And on the new features side of things: - format options now accepts numerical values for all breaks, which will allow you to have more control over output formatting, e.g. format: {breaks: {afterComment: 2}} means clean-css will add two line breaks after each comment	- a new batch option (defaults to false) is added, when set to true it will process all inputs, given either as an array or a hash, without concatenating them. What’s new in version 4.2 clean-css 4.2 introduces the following changes / features: - Adds process method for compatibility with optimize-css-assets-webpack-plugin; - new transition property optimizer;	- preserves any CSS content between /* clean-css ignore:start */ and /* clean-css ignore:end */ comments; - allows filtering based on selector in transform callback, see example; - adds configurable line breaks via format: { breakWith: 'lf' } option.	What’s new in version 4.1 clean-css 4.1 introduces the following changes / features: - inline: false as an alias to inline: ['none']; - multiplePseudoMerging compatibility flag controlling merging of rules with multiple pseudo classes / elements; - removeEmpty flag in level 1 optimizations controlling removal of rules and nested blocks;
17	18	19	20
24	23	22	21
changes, and settings being persisted across sessions. Important: 4.0 breaking changes clean-css 4.0 introduces some breaking changes: - API and CLI interfaces are split, so API stays in this repository while CLI moves to clean-css-cli; - root, relativeTo, and target options are replaced by a single rebaseTo option - this means that rebasing URLs and import inlining	- skipProperties flag in level 2 optimizations controlling which properties won't be optimized; - new animation shorthand and animation-* longhand optimizers; - removeunusedAtRules level 2 optimization controlling removal of unused @counter-style, @font-face, @keyframes, and @namespace at rules; - the web interface gets an improved settings panel with "reset to defaults", instant option	- selectorsSortingMethod level 1 optimization allows false or 'none' for disabling selector sorting; - fetch option controlling a function for handling remote requests; - new font shorthand and font-* longhand optimizers; - removal of optimizeFont flag in level 1 optimizations due to new font shorthand optimizer;	- removeEmpty flag in level 2 optimizations controlling removal of rules and nested blocks; - compatibility: { selectors: { mergeLimit: <number> } } flag in compatibility settings controlling maximum number of selectors in a single rule; - minify method improved signature accepting a list of hashes for a predictable traversal;
is much simpler but may not be (YMMV) as powerful as in 3.x; - debug option is gone as stats are always provided in output object under stats property; - roundingPrecision is disabled by default; - roundingPrecision applies to all units now, not only px as in 3.x; - processImport and processImportFrom are merged into inline option which defaults to local. Remote @import rules are NOT inlined by default anymore;	- splits inliner: { request: ..., timeout: ... } option into inlineRequest and inlineTimeout options; - remote resources without a protocol, e.g. //fonts.googleapis.com/css?family=Domine:700, are not inlined anymore; - changes default Internet Explorer compatibility from 9+ to 10+, to revert the old default use { compatibility: 'ie9' } flag;	- renames keepSpecialComments to specialComments; - moves roundingPrecision and specialComments to level 1 optimizations options, see examples; - moves mediaMerging, restructuring, semanticMerging, and shorthandCompacting to level 2 optimizations options, see examples below; - renames shorthandCompacting option to mergeIntoShorthands;	- level 1 optimizations are the new default, up to 3.x it was level 2; - keepBreaks option is replaced with { format: 'keep-breaks' } to ease transition; - sourceMap option has to be a boolean from now on - to specify an input source map pass it a 2nd argument to minify method or via a hash instead; - aggressiveMerging option is removed as aggressive merging is replaced by smarter override merging.
25	26	27	28
32	31	30	29
- sourceMap - controls whether an output examples; - sourceMapInlinesources - controls embedding sources inside a source maps source map is built; defaults to false; - sourceContent field, defaults to false. defaults to false; see promise interface for examples;	- level - controls optimization level used; defaults to 1; see optimization levels for examples; - rebase - controls URL rebasing; defaults to false; - rebaseTo - controls a directory to which all URLs are rebased, most likely the directory under which the output file will live; defaults to the current directory; - returnPromise - controls whether minify method returns a Promise object or not;	- format - controls output CSS formatting; defaults to false; see formatting options for examples; - inline - controls @import inlining rules; defaults to 'local'; see inlining options for examples; - inlineRequest - controls extra options for inlining remote @import rules, can be any of HTTP(S) request options; - inlineTimeout - controls number of milliseconds after which inlining a remote @import fails; defaults to 5000;	- clean-css constructor accepts a hash as a parameter with the following options available: - compatibility - controls compatibility mode used; defaults to ie10+; see compatibility modes for examples; - fetch - controls a function for handling remote requests; see fetch option for examples (since 4.1.0);

Compatibility modes There is a certain number of compatibility mode shortcuts, namely: - new CleanCSS({ compatibility: '*' }) (default) - Internet Explorer 10+ compatibility mode - new CleanCSS({ compatibility: 'ie9' }) - Internet Explorer 9+ compatibility mode 33	- new CleanCSS({ compatibility: 'ie8' }) - Internet Explorer 8+ compatibility mode - new CleanCSS({ compatibility: 'ie7' }) - Internet Explorer 7+ compatibility mode Each of these modes is an alias to a fine grained configuration, with the following options available: <pre>new CleanCSS({ compatibility: {</pre> 34	<pre>colors: { hexAlpha: false, // controls 4- and 8-character hex color support opacity: true // controls `rgba()` / `hsla()` color support }, properties: { backgroundClipMerging: true, // controls background-clip merging into shorthand</pre>
---	--	---

35

<pre> new CleanCSS({ format: 'keep-breaks' // formats output the default way but adds line breaks for improved readability }) however format option also accept a fine- grained set of options: new CleanCSS({ format: { </pre>	<pre> breaks: { // controls where to insert breaks afterAtRule: false, // controls if a line break comes after an at-rule; e.g. '@charset'; defaults to `false` afterBlockBegins: false, // controls if a line break comes after a block begins; e.g. '@media'; defaults to `false` </pre>	<pre> afterBlockEnds: false, // controls if a line break comes after a block ends, defaults to `false` afterComment: false, // controls if a line break comes after a comment; defaults to `false` afterProperty: false, // controls if a line break comes after a property; defaults to `false` </pre>	<pre> afterRuleBegins: false, // controls if a line break comes after a rule begins; defaults to `false` afterRuleEnds: false, // controls if a line break comes after a rule ends; defaults to `false` beforeBlockEnds: false, // controls if a line break comes before a block ends; defaults to `false` </pre>
49	50	51	52
55	55	54	53
<pre> beforeValue: false // controls if a space comes before a value; e.g. `width: 1rem`; defaults to `false` }, wrapAt: false, // controls maximum line length; defaults to `false` </pre>	<pre> aroundSelectorRelation: false, // controls if spaces come around selector relations; e.g. `div > a`; defaults to `false` beforeBlockBegins: false, // controls if a space comes before a block begins; e.g. `block {`; defaults to `false` </pre>	<pre> indentBy: 0, // controls number of characters to indent with; defaults to `0` indentWith: 'space', // controls a character to indent with, can be `space` or `tab`; defaults to `space` spaces: { // controls where to insert spaces </pre>	<pre> betweenSelectors: false // controls if a line break comes between selectors; defaults to `false` breakWith: '\n', // controls the new line character, can be '\r\n' or '\n' (alias as 'windows' and 'unix', or 'crlf' and '\r\n'); defaults to system one, so former on windows and latter on Unix </pre>
<pre> semicolonAfterLastProperty: false // controls removing trailing semicolons in rule; defaults to `false` - means remove } }) Also since clean-css 5.0 you can use numerical values for all line breaks, which will repeat a line break that many times, e.g: </pre>	<pre> new CleanCSS({ format: { breaks: { afterAtRule: 2, afterBlockBegins: 1, // 1 is synonymous with `true` afterBlockEnds: 2, afterComment: 1, afterProperty: 1, afterRuleBegins: 1, afterRuleEnds: 1, beforeBlockEnds: 1, </pre>	<pre> betweenSelectors: 0 // 0 is synonymous with `false` } }) which will add nicer spacing between at rules and blocks. Inlining options inline option whitelists which @import rules will be processed, e.g. </pre>	<pre> new CleanCSS({ inline: ['local'] // default; enables local inlining only }) new CleanCSS({ inline: ['none'] // disables all inlining }) // introduced in clean-css 4.1.0 new CleanCSS({ </pre>
57	58	59	60
64	63	62	61
<pre> Level 0 optimizations simply means "no optimizations". Use it when you'd like to inline imports and / or rebase URLs but skip everything else. Level 0 optimizations Please note that level 1 optimization options are generally safe while level 2 optimizations should be safe for most users. </pre>	<pre> Optimization levels The level option can be either 0, 1 (default), or 2, e.g. new CleanCSS({ level: 2 }) or a fine-grained configuration given via a hash. </pre>	<pre> new CleanCSS({ inline: ['local', 'remote', 'i fonts.googleapis.com'] // enables all inlining but from given remote source }) new CleanCSS({ inline: ['local', 'remote', 'i fonts.googleapis.com'] // enables local inlining plus given remote source }) </pre>	<pre> new CleanCSS({ inline: false // disables all inlining (alias to `['none']`) }) new CleanCSS({ inline: ['all'] // enables all inlining, same as ['local', 'remote'] }) </pre>

Level 1 optimizations Level 1 optimizations (default) operate on single properties only, e.g. can remove units when not required, turn rgb colors to a shorter hex representation, remove comments, etc Here is a full list of available options: new CleanCSS({ level: { 1: { 65	cleanupCharsets: true, // controls `@charset` moving to the front of a stylesheet; defaults to `true` normalizeUrls: true, // controls URL normalization; defaults to `true` optimizeBackground: true, // controls `background` property optimizations; defaults to `true` 66	optimizeBorderRadius: true, // controls `border-radius` property optimizations; defaults to `true` optimizeFilter: true, // controls `filter` property optimizations; defaults to `true` optimizeFont: true, // controls `font` property optimizations; defaults to `true` 67	optimizeFontWeight: true, // controls `font-weight` property optimizations; defaults to `true` optimizeOutline: true, // controls `outline` property optimizations; defaults to `true` removeEmpty: true, // controls removing empty rules and nested blocks; defaults to `true` 68
72 selectorsSortingMethod: 'standard', // denotes selector sorting method; can be `natural` or 'standard', `none`, or false (the last two since 4.1.0): defaults to 'standard' specialComments: 'all', // denotes a number of /*! */ comments preserved; defaults to `all` 73	71 roundingPrecision: false, // rounds pixel values to `N` decimal places; `false` disables rounding; defaults to `false` 74	70 replaceMultipliers: true, // controls removing redundant zeros; defaults to `true` replaceTimeUnits: true, // controls replacing time units with shorter values; defaults to `true` replaceZeroUnits: true, // controls replacing zero values with units; defaults to `true` 75	69 removeNegativePadding: true, // controls removing negative padding; defaults to `true` removeQuotes: true, // controls removing quotes when unnecessary; defaults to `true` removeWhitespace: true, // controls removing unused whitespace; defaults to `true` 76
tidyAtRules: true, // controls at-rules (e.g. `@charset`, `@import`) optimizing; defaults to `true` tidyBlockScopes: true, // controls block scopes (e.g. `@media`) optimizing; defaults to `true` tidySelectors: true, // controls selectors optimizing; defaults to `true`, 73	variableValueOptimizers: [] // controls value optimizers which are applied to variables } } }); There is an all shortcut for toggling all options at the same time, e.g. new CleanCSS({ level: { 1: { 74	all: false, // set all values to `false` tidySelectors: true // turns on optimizing selectors } }); 75	Level 2 optimizations Level 2 optimizations operate at rules or multiple properties level, e.g. can remove duplicate rules, remove properties redefined further down a stylesheet, or restructure rules by moving them around. Please note that if level 2 optimizations are turned on then, unless explicitly disabled, level 1 optimizations are applied as well. Here is a full list of available options: 76
78 removeDuplicateFontRules: true, // controls duplicate `@font-face` removing; defaults to true removeDuplicateMediaBlocks: true, // controls duplicate `@media` removing; defaults to true removeDuplicateRules: true, // controls duplicate rules removing; defaults to true 79	79 overrideProperties: true, // controls property overriding based on understandability; defaults to true removeEmpty: true, // controls removing empty rules and nested blocks; defaults to `true` reduceNonAdjacentRules: true, // controls non-adjacent rule reducing; defaults to true 80	78 mergeMedia: true, // controls `@media` merging; defaults to true mergeNonAdjacentRules: true, // controls non-adjacent rule merging; defaults to true mergeSemantically: false, // controls semantic merging; defaults to false 81	77 new CleanCSS({ level: { 2: { mergeAdjacentRules: true, // controls adjacent rules merging; defaults to true mergeIntoShorthands: true, // controls merging properties into shorthands; defaults to true 82

<pre>removeUnusedAtRules: false, // controls unused at rule removing; defaults to false (available since 4.1.0) restructureRules: false, // controls rule restructuring; defaults to false skipProperties: [] // controls which properties won't be optimized, defaults to ``[]`` which means all will be optimized (since 4.1.0)</pre>	<pre> } } }); There is an all shortcut for toggling all options at the same time, e.g. new CleanCSS({ level: { 2: { all: false, // sets all values to `false`</pre>	<pre>removeDuplicateRules: true // turns on removing duplicate rules } } }); Plugins In clean-css version 5 and above you can define plugins which run alongside level 1 and level 2 optimizations, e.g.</pre>	<pre>var myPlugin = { level1: { property: function removeRepeatedBackgroundRepeat(_rule property, _options) { // So `background-repeat:no-         repeat no-repeat` becomes `background-repeat:no-repeat`</pre>
81	82	83	84
88	87	86	85
<pre>console.log(output.styles); // optimized output CSS as a string console.log(output.sourceMap); // output source map if requested with `sourceMap` option console.log(output.errors); // a list of errors raised console.log(output.warnings); // a list of warnings raised</pre>	<pre>var output = new CleanCSS(options).minify(source); Once configured clean-css provides a minify method to optimize a given CSS, e.g. Minify method</pre>	<pre>new CleanCSS({plugins: [myPlugin]}) Search test\module-test.js for plugins or check out lib/optimizer/level-1/ property-optimizers and lib/optimizer/ level-1/value-optimizers for more examples. Important: To rewrite your old transform as a plugin, check out this commit.</pre>	<pre>if (property.name == 'background-repeat' && property.value.length == 2 && == property.value[0][1] property.value[1][1]) { property.value.pop(); property.dirty = true; }</pre>
<pre>console.log(output.stats.originalSize); // original content size after import inlining console.log(output.stats.minifiedSize); // optimized content size</pre>	<pre>console.log(output.stats.timeSpent); // time spent on optimizations in milliseconds console.log(output.stats.encyency); // `(originalSize -   minifiedSize) / originalSize`, e.g. 0.25 if size is reduced from 100 bytes to 75 bytes Example: Minifying a CSS string: const CleanCSS = require("clean-css");</pre>	<pre>const output = new CleanCSS().minify(`   a {     color: blue;   }   div {     margin: 5px   } `); console.log(output);</pre>	<pre>// Log: { styles: 'a{color:#00f} div{margin:5px}', stats: { efficiency: 0.6704545454545454, minifiedSize: 29, originalSize: 88, timeSpent: 6 }, errors: [],</pre>
89	90	91	92
96	95	94	93
<pre>new CleanCSS({ returnPromise: true }) .minify(source) .then(function (output) { console.log(output.styles); }) .catch(function (error) { // deal with errors }); CLI utility Clean-css has an associated command line utility that can be installed separately using npm install clean-css-cli. For more</pre>	<pre>(if you won't enclose the path in an array, it will be treated as a CSS source instead). There are several ways to optimize multiple files at the same time, see How to optimize multiple files?. Promise interface If you prefer clean-css to return a Promise object then you need to explicitly ask for it, e.g.</pre>	<pre>new CleanCSS(options).minify(source, function (error, output) { // `output` is the same as in the synchronous call above }); To optimize a single file, without reading it first, pass a path to it to minify method as follows: var output = new CleanCSS(options).minify(['path/ to/file.css'])</pre>	<pre>The minify method also accepts an input source map, e.g. var output = new CleanCSS(options).minify(source, inputSourceMap); or a callback invoked when optimizations are finished, e.g.</pre>

detailed information, please visit <https://github.com/clean-css/clean-css-cli>.

FAQ

How to optimize multiple files?

It can be done either by passing an array of paths, or, when sources are already available, a hash or an array of hashes:

```
new CleanCSS().minify(['path/to/file/
    one', 'path/to/file/two']);
```

```
new CleanCSS().minify({
  'path/to/file/one': {
    styles: 'contents of file one'
  },
  'path/to/file/two': {
    styles: 'contents of file two'
  }
});
```

```
new CleanCSS().minify([
  {'path/to/file/one': {styles:
                        'contents of file one'}}],
```

```
{'path/to/file/two': {'styles':
    'contents of file two'}}
]);
```

Passing an array of hashes allows you to explicitly specify the order in which the input files are concatenated. Whereas when you use a single hash the order is determined by the [traversal order of object properties](#) - available since 4.1.0.

Important note - any `@import` rules already present in the hash will be resolved in memory.

How to apply arbitrary transformations to CSS properties?

Please see [plugins](#).

How to specify a custom rounding precision?

The level 1 roundingPrecision optimization option accept a string with per-unit rounding precision settings, e.g.

How to work with source maps?

- when rebateTo is used with any of above two.

How to process remote imports correctly?

In order to inline remote `@import` statements you need to provide a callback to `minify` method as fetching remote assets is an asynchronous operation, e.g.:

How to optimize a stylesheet with custom rpx units?

Since rpx is a non standard unit (see [#1074](#)), it will be dropped by default as an invalid value.

However you can treat rpx units as regular ones:

```
new CleanCSS({
  compatibility: {
    customUnits: {
```

How to preserve a comment block?

Use the $/*$ notation instead of the standard one $/$ $*$:

Optimizing this CSS will result in the following output:

```
.block-1{color:red}
.block-special
{
color: transparent
}
.block-2{margin:0}
```

```

    {
      color: red
    }
    block-1
  {
    color: red
  }
  /* clean-css ignore:start */
  {
    block-special
  }
  color: transparent
}
/* clean-css ignore:end */
}
block-2
{
  margin: 0
}

```

```

    rpx: true
  }
}
}).minify(source)

```

How to keep a CSS fragment intact?

Note: available since 4.2.0.

Wrap the CSS fragment in special comments which instruct clean-css to preserve it, e.g.

<pre>// access output.sourceMap for SourceMapGenerator object // see https://github.com/mozilla/ source-map/#sourcemapgenerator for more details }); You can also pass an input source map directly as a 2nd argument to minify method:</pre>	<pre>new CleanCSS({ sourceMap: true, rebaseTo: pathToOutputDirectory }) .minify(source, inputSourceMap, function (error, output) { // access output.sourceMap to access SourceMapGenerator object // see https://github.com/mozilla/ source-map/#sourcemapgenerator for more details });</pre>	or even multiple input source maps at once: <pre>new CleanCSS({ sourceMap: true, rebaseTo: pathToOutputDirectory }).minify({ 'path/to/source/1': { styles: '...styles...', sourceMap: '...source-map...' }, 'path/to/source/2': { styles: '...styles...', sourceMap: '...source-map...' } }</pre>	<pre>}, function (error, output) { // access output.sourceMap as above }); How to apply level 1 & 2 optimizations at the same time? Using the hash configuration specifying both optimization levels, e.g. new CleanCSS({ level: { 1: {</pre>
<pre>113</pre>	<pre>114</pre>	<pre>115</pre>	<pre>116</pre>
<pre>120</pre>	<pre>119</pre>	<pre>118</pre>	<pre>117</pre>
<pre>121</pre>	<pre>122</pre>	<pre>123</pre>	<pre>124</pre>
<pre>128</pre>	<pre>127</pre>	<pre>126</pre>	<pre>125</pre>

<pre>], inlinedStylesheets: [], warnings: [] } </pre> Clean-css for Gulp An example of how you can include clean-css in gulp <pre> const { src, dest, series } = require('gulp'); const CleanCSS = require('clean-css'); </pre>	<pre> const concat = require('gulp-concat'); function css() { const options = { compatibility: '*', // (default) - Internet Explorer 10+ compatibility mode inline: ['all'], // enables all inlining, same as ['local', 'remote'] }; level: 2 // Optimization levels. The level option can be either 0, 1 (default), or 2, e.g. // Please note that level 1 optimization options are generally safe while level 2 optimizations should be safe for most users. return src('app/**/*.css') } </pre>	<pre> .pipe(concat('style.min.css')) .on('data', function(file) { const buferFile = new CleanCSS(options).minify(file.content return file.contents = Buffer.from(buferFile.styles) }) .pipe(dest('build')) } exports.css = series(css) </pre>	
129	130	131	132
136	then install dependencies: <pre> git clone git@github.com:clean-css/ clean-css.git </pre> First clone the sources: Contributing See CONTRIBUTING.md. How to get started? Verify your copy: <pre> cd clean-css npm install </pre> then use any of the following commands to	https://clean-css.github.io/ (official web interface) - http://refresh-sf.com/ - http://adamburgess.github.io/clean-css-online/ How to use clean-css from web browser? Metalsmith: metalsmith-clean-css Lasso: lasso-clean-css Start: start-clean-css	There is a number of 3rd party plugins to popular build tools: Broccoli: broccoli-clean-css Brunch: clean-css-brunch Grunt: grunt-contrib-cssmin Gulp: gulp-clean-css Gulp: using vinyl-map as a wrapper - courtesy of @sodoko component-builder: builder-clean-css How to use clean-css with build tools?
<pre> npm run bench # for clean-css benchmarks (see [test/ bench.js](https://github.com/ clean-css/clean-css/blob/ master/test/bench.js) for details) npm run browserify # to create the browser-ready clean-css version </pre>	<pre> npm run check # to lint JS sources with [JSHint](https:// github.com/jshint/jshint/) npm test # to run all tests </pre>	Acknowledgments Sorted alphabetically by GitHub handle: @abarre (Anthony Barre) for improvements to @import processing; @alexlamsl (Alex Lam S.L.) for testing early clean-css 4 versions, reporting bugs, and suggesting numerous improvements. @altschuler (Simon Altschuler) for fixing @import processing inside comments;	@ben-eb (Ben Briggs) for sharing ideas about CSS optimizations; @davisjam (Jamie Davis) for disclosing ReDOS vulnerabilities; @facelessuser (Isaac) for pointing out a flaw in clean-css’ stateless mode; @grandrath (Martin Grandrath) for improving minify method source traversal in ES6; @jmalonzo (Jan Michael Alonzo) for a patch removing node.js’ old sys package;
137	138	139	140
144	License clean-css is released under the MIT License.	@venemo (Timur Kristóf) for an outstanding contribution of advanced property optimizer for 2.2 release; @vvo (Vincent Voyer) for a patch with better empty element regex and for inspiring us to do many performance improvements in 0.4 release; @xhmkosr for suggesting new features, like option to remove special comments and strip out URLs quotation, and pointing out numerous improvements like JSHint, media queries, etc.	@lukeapage (Luke Page) for suggestions and testing the source maps feature; Plus everyone else involved in #125 for pushing it forward; @madwizard-thomas for sharing ideas about @import inlining and URL rebasing. @ngyikp (Ng Yik Phang) for testing early clean-css 4 versions, reporting bugs, and suggesting numerous improvements. @wagene (Peter Wagene) for suggesting improvements to @import inlining behavior;
	143	142	141